

CMSC 471 Programming Assignment 1

Name: Matt Nyandjo

Section: M & W

Question 1: Heuristic Design and Admissibility

Steps heuristic

Pseudocode

```
estimate = 0
FOR each position in the word:
    IF current letter != goal letter:
        estimate = estimate + 1
RETURN estimate
```

Written explanation

First, start with a total of zero. Next, compare the current word with the goal word one letter position at a time. Whenever the two letters at a position are different, add one to the total. Finally, after every position has been checked, use the total as the heuristic estimate.

Why it works

Each incorrect position must be changed at least once, and one action changes only one position. Therefore, the number of incorrect positions is a minimum remaining cost under the steps measure. The heuristic returns 0 at the goal and cannot overestimate the true cost.

Scrabble heuristic

Pseudocode

```
estimate = 0
FOR each position in the word:
    IF current letter != goal letter:
        goal_letter = goal letter at this position
        estimate = estimate + Scrabble value of goal_letter
RETURN estimate
```

Written explanation

First, start with a total of zero. Next, compare the current word with the goal word one letter position at a time. Whenever the letters are different, identify the letter needed in the goal word at that position and add that letter's Scrabble value to the total. Finally, after every position has been checked, use the total as the heuristic estimate.

Why it works

Every incorrect position must eventually receive its goal letter. The heuristic adds the Scrabble cost of each of those required letters. A solution may make additional substitutions, but those only add cost, so this estimate cannot be greater than the true remaining cost. It returns 0 at the goal.

Frequency heuristic

Pseudocode

```
IF current word == goal word:  
    RETURN 0  
estimate = rarity value of the goal word  
FOR each position in the word:  
    IF current letter != goal letter:  
        estimate = estimate + 1  
RETURN estimate
```

Written explanation

First, check whether the current word already matches the goal word. If it does, use zero and stop. Otherwise, start the total with the rarity value of the goal word. Next, compare the two words one letter position at a time. Whenever the letters are different, add one to the total. Finally, after every position has been checked, use the total as the heuristic estimate.

Why it works

Each incorrect position requires at least one action, and every successful path must enter the goal word and pay its rarity cost. The heuristic counts only those unavoidable costs, so it cannot overestimate the true remaining frequency cost.

Question 2: Does the Heuristic Actually Help?

Experimental results

Word pair	Cost	UCS cost	UCS exp.	A* cost	A* exp.	Reduction
WHY -> ASK	Steps	5.00	773	5.00	16	97.93%
WHY -> ASK	Scrabble	9.00	765	9.00	13	98.30%
WHY -> ASK	Frequency	38.86	808	38.86	634	21.53%
BEAR -> DUCK	Steps	4.00	691	4.00	6	99.13%
BEAR -> DUCK	Scrabble	11.00	2,981	11.00	6	99.80%
BEAR -> DUCK	Frequency	38.24	1,084	38.24	302	72.14%

Answers

1. UCS and A* always returned the same optimal cost in all six experiments. They may return different optimal paths when ties exist, but there was no cost discrepancy.
2. For WHY -> ASK, Scrabble produced the largest reduction at 98.30%. For BEAR -> DUCK, Scrabble also produced the largest reduction at 99.80%.
3. For both word pairs, frequency was the least informative heuristic. Its reduction was 21.53% for WHY -> ASK and 72.14% for BEAR -> DUCK.
4. The steps and Scrabble heuristics were both effective because they counted unavoidable costs. Scrabble had the largest percentage reductions partly because its UCS baseline was higher; for BEAR

-> DUCK, both heuristics expanded only 6 A* nodes. Frequency was least informative because it ignored the rarity costs of intermediate words, which make up much of the total path cost.

Question 3: Reversibility and Direction-Dependent Costs

Steps: Yes

Every legal action is reversible, and both directions of every edge cost 1. Reversing any path therefore preserves its cost. The reverse of an optimal W1-to-W2 path is a W2-to-W1 path with the same cost, and the same argument in the other direction proves equality of the two optimal costs.

Scrabble: No

Counterexample: dog and cog are both dictionary words and differ in one position. The optimal cost dog -> cog is 3 because the move inserts c. The optimal cost cog -> dog is 2 because the reverse move inserts d. The direct action is optimal in each direction: any forward solution must eventually insert the destination letter in the differing position, so it cannot cost less than that direct insertion. Action reversibility therefore does not imply cost symmetry.

Frequency: No

Using the same adjacent pair, moving from dog to cog costs 1 plus the rarity value of cog: $1 + 11.393166 = 12.393166$. Moving from cog to dog costs 1 plus the rarity value of dog: $1 + 5.819885 = 6.819885$. Each direct action is optimal because every route must eventually enter its destination and pay at least that destination's entry cost. The action is reversible, but its cost depends on the word being entered.

Question 4: LLM Heuristic Audit

Model and exact prompt

Model used: OpenAI GPT-5.6 Sol.

Exact prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Model proposal

The model proposed adding the goal word's rarity to the baseline at every non-goal state because entering the goal is unavoidable.

$$h_1(n) = 0 \text{ if } n \text{ is the goal; otherwise } h_0(n) + r(\text{goal})$$

Why the proposal works

The proposal is admissible because it only counts costs that every solution must pay. Since one move changes only one letter, every incorrect position needs at least one move. Each move has a base cost of 1. The final move must also enter the goal word, so the goal word's rarity value must be paid. This means the estimate cannot be greater than the actual remaining cost. Once the goal is reached, the estimate is 0.

It is also more informative than the original heuristic. The original heuristic only counts incorrect letter positions. The new heuristic includes that same count and adds the goal word's rarity value before the goal is reached. All rarity values in the supplied dictionary are greater than 0, so the new estimate gives A* more information while remaining safe. Therefore, the proposal does not need to be changed.

Experimental comparison with h0

Word pair	Heuristic	Optimal cost	Nodes expanded	Reduction vs. h0
WHY -> ASK	h0	38.85540652	755	—
WHY -> ASK	h1	38.85540652	634	16.03%
BEAR -> DUCK	h0	38.2402284	720	—
BEAR -> DUCK	h1	38.2402284	302	58.06%

Both heuristics preserved the same optimal costs. On these two problems, the rarity-aware h1 reduced expansions, so the evidence supports using it. This experimental evidence measures usefulness; the proof above, rather than the experiments alone, establishes admissibility.

AI-use disclosure

I used OpenAI GPT-5.6 Sol to help debug [dc.py](#) giving example code snippets. I also used it to clarify my explanations for Questions 1–3. I verified the work using the supplied tests and by comparing the UCS and A* results.